

Práctica 2: Estructuras de Datos para la Inteligencia Artificial

Materia: Estructura de datos

**Hecho por: Ashley Dayanne Rodríguez
Acosta**

Introducción

En el presente documento se desarrollan tres ejercicios prácticos que ejemplifican la aplicación de **estructuras de datos estáticas** en problemas de inteligencia artificial y programación. Estos ejemplos permiten reconocer la importancia de los arreglos y matrices como herramientas fundamentales para almacenar, manipular y procesar información de manera eficiente.

Ejercicio 1: Vector de Características para Reconocimiento de Patrones

Problema: Un modelo de reconocimiento de patrones utiliza un vector de características para identificar objetos. Crea un arreglo unidimensional para representar las características de una flor (ej., [3.5, 1.4, 0.2], que podría ser la longitud del sépalo, el pétalo y la anchura del pétalo).

Objetivo: El programa debe normalizar el vector de características, dividiendo cada valor por la suma total del vector. Esto es un paso común en la preparación de datos para el entrenamiento de modelos de IA.

Conceptos a reforzar: Declaración de arreglos, iteración, operaciones matemáticas y normalización de datos.

Algoritmo:

1. Declarar el vector de características.
2. Calcular la suma total de los valores.
3. Recorrer el vector dividiendo cada valor entre la suma.
4. Mostrar el vector normalizado.

```
# Reconocimiento de patrones
caracteristicas = [3.5, 1.4, 0.2]
suma = 0
for i in caracteristicas:
    suma += i
print("La suma de las características es: ", suma)

for i in range(len(caracteristicas)):
    caracteristicas[i] = caracteristicas[i]/suma # Normalización
print("Las características normalizadas son: ", caracteristicas)
```

Conclusión: Me ayuda a entender el reconocimiento de patrones y normalización de características.

Ejercicio 2: Matriz de Conexiones para una Red Neuronal Simple

Problema: Una red neuronal simple tiene 3 neuronas de entrada y 2 de salida. Crea una matriz de 3x2 para representar los "pesos" o conexiones entre las neuronas.

Objetivo: El programa debe tomar un vector de entrada (un arreglo de 3 elementos) y realizar el producto punto con la matriz de pesos para obtener los valores de salida.

Conceptos a reforzar: Declaración y manipulación de matrices, producto punto (una operación clave en IA) y manejo de vectores.

Algoritmo:

1. Declarar el vector de entrada.
2. Declarar la matriz de pesos.
3. Multiplicar cada entrada por el peso correspondiente y acumularlo en la salida.
4. Mostrar los valores de salida.

```
# Entrenamiento de una red neuronal
entrada = [1, 2, 3]

# Matriz pesos 3x2
pesos = [
    [4, 1],
    [6, 3],
    [4, 2]
]

salida = [0, 0]

for j in range(len(pesos[0])):
    for i in range(len(entrada)):
        salida[j] += entrada[i] * pesos[i][j]

print("Matriz Entrada", entrada)
print("Matriz pesos", pesos)
print("Matriz salida", salida)
```

Conclusión: Me ayuda a comprender y poder utilizar las matrices en el cálculo de salidas mediante multiplicaciones de entrada y pesos.

Ejercicio 3: Mapa de Riesgo para la Navegación de un Vehículo Autónomo

Problema: Un vehículo autónomo utiliza un mapa de riesgo para tomar decisiones de navegación. Representa el mapa como una matriz de 8x8 donde 0 es un área segura, 1 un área de precaución y 2 un área de alto riesgo.

Objetivo: El programa debe imprimir el mapa de riesgo. Luego, debe contar el total de áreas de precaución (1) y de alto riesgo (2). Por último, debe cambiar todos los valores de alto riesgo a áreas de precaución para simular una actualización del mapa.

Conceptos a reforzar: Matrices, bucles anidados, contadores y actualización de elementos en una matriz.

Algoritmo:

1. Crear la matriz de 8x8 con valores 0, 1 y 2.
2. Imprimir la matriz inicial.
3. Recorrer la matriz contando las áreas de precaución y riesgo.
4. Cambiar los valores de 2 a 1.
5. Mostrar la matriz actualizada y el nuevo conteo.

```
# Mapa de riesgo para la navegación de un vehículo autónomo

# Crear una matriz de 8x8
matriz = [
    [0, 1, 2, 1, 2, 0, 1, 1],
    [0, 1, 2, 1, 2, 0, 1, 1],
    [0, 1, 2, 1, 2, 0, 1, 1],
    [0, 1, 2, 1, 2, 0, 1, 1],
    [0, 1, 2, 1, 2, 0, 1, 1],
    [0, 1, 2, 1, 2, 0, 1, 1],
    [0, 1, 2, 1, 2, 0, 1, 1],
    [0, 1, 2, 1, 2, 0, 1, 1]
]

for i in range(len(matriz)):
    for j in range(len(matriz[i])):
        print(matriz[i][j], end=" ")
    print()

area_precaucion = 0 #1
area_riesgo = 0 #2

for i in range(len(matriz)):
    for j in range(len(matriz[i])):
        if matriz[i][j] == 1:
            area_precaucion += 1
        if matriz[i][j] == 2:
            area_riesgo += 1
    print()

print("Area de precaución (1): {area_precaucion}")
print("Area de riesgo (2): {area_riesgo}")

# Cambiar todos los 2 por 1
print("Actualización de matriz de navegación")

for i in range(len(matriz)):
    for j in range(len(matriz[i])):
        if matriz[i][j] == 2:
            matriz[i][j] = 1
    print()

for i in range(len(matriz)):
    for j in range(len(matriz[i])):
```

```

        print(matriz[i][j], end="")
    print()

    area_precaucion_actualizada = 0
    area_riesgo_actualizada = 0

    for i in range(len(matriz)):
        for j in range(len(matriz[i])):
            print(matriz[i][j], end="")
            if matriz[i][j] == 1:
                area_precaucion_actualizada += 1
            if matriz[i][j] == 2:
                area_riesgo_actualizada += 1
        print()

    print("Area de precaución (1): {area_precaucion_actualizada}")
    print("Area de riesgo (2): {area_riesgo_actualizada}")

```

Conclusión: Refuerza mi conocimiento sobre como las matrices permiten modelar un entorno, analizar riesgos y actualizar información en tiempo real.